

AI-Checker

AI Content Detection — Proof of Concept

Technical Documentation

Document Version	1.0
Date	July 2026
Prepared By	Sadman Fahim — Software Engineer
Idea & Direction	Roy Kumar — Team Lead
Recommended By	MD Shah Ishtiaque
Organization	NeoNexor Software
Classification	Internal — Confidential

This document contains proprietary information of NeoNexor Software. It is intended solely for internal use and must not be distributed, reproduced, or disclosed to any third party without prior written authorization.

Table of Contents

1. Executive Summary
2. Project Overview
3. System Architecture
4. Tech Stack
5. Project Structure
6. Fallback Architecture
7. Provider Configuration
8. API Specification
9. Video Processing Pipeline
10. Frontend Architecture
11. Environment Variables
12. Setup Guide
13. Error Handling Strategy
14. Security Considerations
15. Design Decisions
16. Performance Characteristics
17. Limitations & Scope
18. Future Recommendations

1. Executive Summary

AI-Checker is a proof-of-concept web application developed by NeoNexor Software to address the growing need for AI-generated content detection. The application accepts three input modalities — text, image, and video — and returns a normalized likelihood score (0–100%) indicating whether the content was generated by an AI system.

The system delegates all detection to third-party APIs via their free tiers, eliminating the need for self-hosted machine learning infrastructure. The defining architectural feature is an automatic provider fallback chain: if the primary detection provider fails (due to rate limiting, timeout, or error), the system transparently retries with the next provider in a priority list. The end user receives a result without knowing which provider fulfilled it.

Built with Next.js 16 (App Router), TypeScript (strict mode), and Tailwind CSS, the application represents a modern, type-safe, and maintainable codebase suitable for evaluation and potential production evolution.

Key Outcomes

- Three fully functional detection paths (text, image, video) with live API integration
- Automatic provider fallback chain demonstrated with real-world rate limit handling
- Clean, responsive UI with dark/light theme support and animated result presentation
- Production build passes TypeScript strict checking, ESLint, and Next.js compilation with zero errors
- Complete documentation package covering architecture, API, setup, and design decisions

2. Project Overview

2.1 Objective

Build a web-based tool that analyzes content (text, image, or video) and returns a percentage-based likelihood of AI generation, using free-tier third-party detection APIs with automatic fallback between providers.

2.2 Scope

- Three input types: plain text, image (JPG/PNG), video (MP4)
- One unified output shape: percentage (0–100) + verdict bucket (Likely Human / Uncertain / Likely AI-generated)
- No AI models run on the application server or in the browser
- No user authentication, database, or persistent storage
- Designed for internal demonstration and evaluation purposes

2.3 Constraints

- Image file size limit: 10MB (JPG/PNG only)
- Video file size limit: 20MB (MP4 only)
- Text minimum: 20 characters
- Dependent on third-party API availability and free-tier quotas
- No offline detection capability

3. System Architecture

3.1 High-Level Architecture

The system follows a three-tier architecture: a client-side React UI, server-side Next.js API route handlers, and external third-party detection APIs. The server layer acts as a proxy and orchestration point — it never runs detection models directly.

Layer	Responsibility	Technology
Client (Browser)	User interface, input validation, result display	React 19, Tailwind CSS, Framer Motion
Server (API Routes)	Input validation, provider orchestration, response formatting	Next.js 16 API Routes, Node.js runtime
External APIs	AI content detection analysis	Sapling, Sightengine, GPTZero

3.2 Request Flow

Every detection request follows the same high-level flow regardless of modality:

1. User submits input via the browser (text, image file, or video file)
2. Client validates input (type, size, format) before sending
3. Client sends HTTP POST to the corresponding API route
4. API route handler validates the request server-side
5. Handler calls `detectWithFallback(providers[], input)`
6. Orchestrator tries providers sequentially until one succeeds (8s timeout each)
7. Successful provider returns `aiProbability` (0.0–1.0)
8. Handler converts to percentage, maps to verdict, returns JSON
9. Client renders the result card with animated percentage and gradient bar

3.3 Provider Abstraction

All detection providers implement a uniform interface, decoupling the orchestration logic from specific API implementations. Adding a new provider requires only implementing the interface in a new file and appending it to the provider array in the route handler.

```
interface Provider {
  name: string;
  detect(input: ProviderInput): Promise<DetectionResult>;
}

interface DetectionResult {
  aiProbability: number; // 0..1 normalized
  providerName: string; // e.g. "sapling"
  raw?: unknown;        // original response for debugging
}
```

4. Tech Stack

Category	Technology	Purpose
Framework	Next.js 16 (App Router)	Full-stack React framework with server-side API routes
Language	TypeScript 5.x (strict)	Type-safe development with compile-time error checking
Styling	Tailwind CSS 4.x	Utility-first CSS framework with dark mode support
Animation	Framer Motion	Production-quality React animations and transitions
HTTP Client	Axios	HTTP requests with timeout support and multipart handling
Validation	Zod	Runtime schema validation with TypeScript type inference
File Upload	react-dropzone	Drag-and-drop file upload with type/size validation
Video Processing	fluent-ffmpeg	FFmpeg wrapper for video frame extraction
FFmpeg Binary	@ffmpeg-installer/ffmpeg	Bundled FFmpeg — no system install needed
Charts	Recharts	Declarative React charting for per-frame timeline
Notifications	react-hot-toast	Lightweight toast notifications for error/success
Icons	Lucide React	Tree-shakable icon library
Utilities	clsx	Conditional CSS class composition

5. Project Structure

5.1 Directory Layout

Directory / File	Purpose
app/layout.tsx	Root layout — HTML shell, theme provider, toast container, metadata
app/page.tsx	Main page — hero section, tab switcher, detection panels
app/globals.css	Global styles — CSS variables, theme colors, keyframe animations
app/api/check/text/route.ts	POST /api/check/text — text detection endpoint
app/api/check/image/route.ts	POST /api/check/image — image detection endpoint
app/api/check/video/route.ts	POST /api/check/video — video detection with frame extraction
components/TabSwitcher.tsx	Segmented tab control with animated active indicator
components/TextCheckPanel.tsx	Text input panel — textarea, character count, validation
components/ImageCheckPanel.tsx	Image upload panel — drag-and-drop, preview, validation
components/VideoCheckPanel.tsx	Video upload panel — MP4 upload, frame analysis
components/ResultCard.tsx	Detection result display — animated counter, gradient bar, verdict
components/FrameTimelineChart.tsx	Per-frame score timeline chart (recharts)
components/ThemeProvider.tsx	Theme context — system preference detection, cookie persistence
components/ThemeToggle.tsx	Day/night toggle button with icon animation
lib/fallback.ts	Provider fallback orchestrator — sequential retry with 8s timeout
lib/scoring.ts	Percentage to verdict bucket mapping
lib/videoFrames.ts	FFmpeg frame extraction utility
lib/providers/types.ts	Shared TypeScript interfaces for all providers
lib/providers/text/sapling.ts	Sapling AI — primary text detection provider
lib/providers/text/gptzero.ts	GPTZero — fallback text detection provider
lib/providers/text/zerogpt.ts	ZeroGPT — stubbed (no confirmed API)
lib/providers/image/sightengine.ts	Sightengine — primary image detection provider
lib/providers/image/aiornot.ts	AI or Not — stubbed (no confirmed API)

6. Fallback Architecture

The fallback chain is the core architectural feature of this POC. It ensures that a single provider failure does not result in a user-facing error.

6.1 Algorithm

The `detectWithFallback` function accepts an ordered array of providers and an input. It calls each provider sequentially. On the first success, it returns the result immediately. On failure, it logs the error and moves to the next provider. If all providers fail, it throws an aggregated error.

6.2 Timeout Mechanism

Each provider call is wrapped in a `Promise.race` against a timeout promise. The default timeout is 8 seconds per provider, configurable via the `PROVIDER_TIMEOUT_MS` constant. If a provider does not respond within the timeout, a `ProviderUnavailableError` is thrown and the orchestrator proceeds to the next provider.

6.3 Failure Conditions

Condition	Trigger	Behavior
Rate limit	HTTP 429 from provider	Throw <code>ProviderUnavailableError</code> , try next
Auth failure	HTTP 401 or 403	Throw <code>ProviderUnavailableError</code> , try next
Server error	HTTP 5xx from provider	Throw <code>ProviderUnavailableError</code> , try next
Timeout	No response within 8 seconds	Timeout promise rejects, try next
Schema mismatch	Response missing expected fields	Throw <code>ProviderUnavailableError</code> , try next
All providers fail	Every provider in chain failed	HTTP 503 returned to client

6.4 Observability

Every fallback event is logged via `console.warn` with the provider name and failure reason. This provides a clear audit trail during development and demos. Example log output:

```
[fallback] sapling failed: quota exceeded. Trying next provider...
[fallback] gptzero failed: timeout. Trying next provider...
```

7. Provider Configuration

7.1 Text Detection Providers

Priority	Provider	Endpoint	Status	Free Tier
1	Sapling	api.sapling.ai/api/v1/aidetect	Active	50K chars/day
2	GPTZero	api.gptzero.me/v2/predict/text	Paid only	N/A
3	ZeroGPT	N/A	Stubbed	No confirmed API

7.2 Image Detection Providers

Priority	Provider	Endpoint	Status	Free Tier
1	Sightengine	api.sightengine.com/1.0/check.json	Active	2K ops/month
2	AI or Not	N/A	Stubbed	No confirmed API

7.3 Video Detection

Video detection reuses the image provider chain. The system extracts up to 8 evenly-spaced JPEG frames from the uploaded MP4 using FFmpeg, runs each frame through the image detection fallback chain, then computes the arithmetic mean of per-frame scores as the overall result.

7.4 Adding a New Provider

To integrate a new detection provider, follow these steps:

- Create a new file in `lib/providers/<modality>/` implementing the `Provider` interface
- Handle all API-specific logic (authentication, request format, response parsing) within the file
- Throw `ProviderUnavailableError` for any failure condition
- Add the provider instance to the provider array in the corresponding API route handler
- The fallback orchestrator handles everything else automatically

8. API Specification

8.1 Common Response Format

All successful detection responses share a consistent JSON structure:

```
{
  "percentage": 78,
  "verdict": "Likely AI-generated",
  "provider": "sapling"
}
```

8.2 Verdict Buckets

Percentage Range	Verdict	Interpretation
0–34%	Likely Human	Low probability of AI generation
35–65%	Uncertain	Inconclusive — detection signal is ambiguous
66–100%	Likely AI-generated	High probability of AI generation

8.3 Endpoints

Method	Path	Content-Type	Input	Notes
POST	/api/check/text	application/json	{ "text": "..." }	Min 20 characters
POST	/api/check/image	multipart/form-data	image file field	JPG/PNG, max 10MB
POST	/api/check/video	multipart/form-data	video file field	MP4 only, max 20MB

8.4 Error Responses

HTTP Code	Meaning	Example Message
200	Detection successful	Result JSON with percentage, verdict, provider
400	Invalid input	"Please provide at least 20 characters of text."
503	All providers failed	"All detection providers are currently unavailable."

9. Video Processing Pipeline

Video detection is the most complex path. Since most free-tier detection APIs do not accept video input directly, the system extracts representative frames and analyzes each as an image.

9.1 Processing Steps

1. User uploads an MP4 file (max 20MB, validated client-side and server-side)
2. The file is written to a temporary path on disk (`os.tmpdir()`)
3. FFmpeg extracts 8 evenly-spaced JPEG frames from the video
4. Each frame is read into a Node.js Buffer
5. All 8 frames are sent through `detectWithFallback` in parallel (`Promise.all`)
6. Per-frame scores are collected as an array
7. The overall score is computed as the arithmetic mean of all frame scores
8. All temporary files (video + frames) are cleaned up in a `finally` block
9. The response includes both the aggregate percentage and the per-frame array

9.2 Frame Count Rationale

The default of 8 frames was chosen to balance coverage, API consumption, and processing time. With parallel execution, processing time is approximately equal to a single-frame latency plus overhead. Each frame consumes one Sightengine API operation, keeping free-tier usage sustainable at 8 operations per video analysis.

9.3 Per-Frame Timeline

The `perFrame` array in the API response enables the frontend to render a timeline chart showing how detection confidence varies across the video duration. This provides actionable insight — for example, identifying that a specific segment of the video appears more synthetic than others.

10. Frontend Architecture

10.1 Component Hierarchy

The frontend is a single-page application with a tab-based layout. All state is local to each panel component via React useState. There is no global state management — the POC scope does not warrant Redux, Zustand, or Context for data flow.

Component	Responsibility
TabSwitcher	Segmented tab control — animated active indicator, Lucide icons per tab
TextCheckPanel	Text input with character count, 20-char validation, skeleton loading, result display
ImageCheckPanel	Drag-and-drop upload (react-dropzone), thumbnail preview, 10MB/JPG/PNG validation
VideoCheckPanel	MP4 upload, 20MB validation, frame analysis loading state, timeline chart
ResultCard	Animated percentage counter, green-amber-red gradient bar, verdict, provider attribution, disclaimer
FrameTimelineChart	Recharts LineChart — monotone curve, gradient fill, custom tooltip, themed for light/dark
ThemeProvider	System preference detection, manual toggle, cookie persistence, dark class on HTML
ThemeToggle	Sun/moon icon with rotation animation, reads/toggles theme context

10.2 Theme System

The application supports light and dark themes. Theme preference is detected from the operating system via prefers-color-scheme and can be toggled manually. The selected theme is persisted via a cookie to prevent flash-of-wrong-theme on subsequent page loads. The theme is applied by toggling a dark class on the <html> element.

10.3 Result Presentation

The result card uses an animated counter that counts from 0 to the final percentage, a gradient progress bar that transitions from green (low AI likelihood) through amber (uncertain) to red (high AI likelihood), and a mandatory disclaimer caption that is always visible: *"This is a detection signal, not proof. Treat it as one input among several."*

11. Environment Variables

Variable	Required	Provider	Description
SAPLING_API_KEY	Yes	Sapling	Text detection API key (50K chars/day free)
SIGHTENGINE_API_USER	Yes	Sightengine	Image/video detection API user ID
SIGHTENGINE_API_SECRET	Yes	Sightengine	Image/video detection API secret
GPTZERO_API_KEY	No	GPTZero	Text fallback (paid subscription required)
AIORNOT_API_KEY	No	AI or Not	Image fallback (no confirmed public API)

Security: All keys are stored in `.env.local` (gitignored), read server-side only via `process.env`, and never exposed to the client JavaScript bundle. The application does not crash when keys are missing — the corresponding provider is simply skipped in the fallback chain.

12. Setup Guide

12.1 Prerequisites

Requirement	Minimum Version	Check Command
Node.js	18.17	node --version
npm	9.0	npm --version
Git	2.0	git --version

12.2 Installation Steps

- Step 1:** Clone the repository
`git clone <repository-url>`
`cd ai-checker`
- Step 2:** Install dependencies
`npm install`
- Step 3:** Configure environment variables
`cp .env.example .env.local`
Open `.env.local` and add your API keys.
- Step 4:** Start the development server
`npm run dev`
- Step 5:** Open `http://localhost:3000` in your browser.

12.3 Build Commands

Command	Purpose
<code>npx tsc --noEmit</code>	TypeScript type checking (zero errors expected)
<code>npx eslint .</code>	Linting (zero warnings/errors expected)
<code>npm run build</code>	Production build compilation
<code>npm start</code>	Start production server on port 3000

13. Error Handling Strategy

13.1 Error Handling Layers

Layer	Responsibility	Action on Error
Provider	Catch API-specific errors	Throw ProviderUnavailableError
Orchestrator	Catch provider errors	Log, try next provider in chain
Route Handler	Catch orchestrator failure	Return HTTP 503 with friendly message
Frontend	Catch HTTP errors	Display toast notification to user

13.2 User-Facing Error Policy

The frontend never displays raw error messages, stack traces, or provider-specific details. All errors are translated to user-friendly messages. Provider failures result in: *"Detection is temporarily unavailable. Please try again shortly."* Input validation errors provide specific guidance (e.g., *"Please provide at least 20 characters of text."*).

14. Security Considerations

Concern	Mitigation
API key exposure	Keys in .env.local (gitignored), read server-side only, never in client bundle
File upload abuse	Client-side and server-side validation of file type and size
Temporary file persistence	Video frames written to os.tmpdir(), deleted in finally block
Response injection	Provider responses validated for expected shape before use
CORS	Next.js API routes are same-origin by default
Environment leakage	No NEXT_PUBLIC_ prefix on any secret variable

15. Design Decisions

Third-party APIs over self-hosted models

Eliminates ML infrastructure complexity for a POC. Provider abstraction layer allows migration to self-hosted models later without architectural changes.

Fallback chain over single provider

Free-tier rate limits are easily exhausted. Multiple providers dramatically increase availability. Core differentiator of the POC — demonstrates resilience thinking.

Next.js App Router over Pages Router

Current recommended architecture for new projects. Server-side API routes provide clean proxy pattern for third-party calls without exposing keys.

Server-side proxy over direct client calls

API keys must never reach the browser. Server provides single orchestration point. Some provider APIs have CORS restrictions preventing direct browser calls.

Percentage + verdict over binary yes/no

AI detection is probabilistic. A percentage conveys confidence level, not false certainty. The three-tier verdict preserves nuance while remaining human-readable.

Frame sampling over full video upload

Most free-tier APIs do not accept video. Reuses existing image detection infrastructure. Per-frame scores enable a timeline chart for segment-level insight.

Mandatory disclaimer

Independent testing reports false-positive rates of 5–33% across detectors. Presenting results as definitive could lead to incorrect accusations. The disclaimer protects the team during demos.

No authentication or database

Out of scope for a POC. Adding either later requires no architectural changes — the detection core is completely decoupled from user management or storage.

16. Performance Characteristics

Operation	Expected Latency	Primary Bottleneck
Text detection	1–3 seconds	External API response time
Image detection	2–5 seconds	File upload to provider + API processing
Video detection	10–30 seconds	FFmpeg extraction + 8 parallel API calls
Provider fallback	+1–8 seconds per failure	Timeout wait before trying next provider

Video detection frames are analyzed concurrently via `Promise.all`, reducing total processing time from 8x sequential latency to approximately 1x plus overhead.

17. Limitations & Scope

This is a proof-of-concept with the following intentional constraints:

- No user authentication or account management
- No self-hosted or fine-tuned detection models — all detection delegated to third-party APIs
- Video input restricted to MP4 format only
- File size limits: 10MB for images, 20MB for video
- No persistent storage — detection results are not saved between sessions
- Detection accuracy is entirely dependent on third-party provider quality
- Free-tier API quotas are limited — heavy usage may exhaust daily/monthly limits
- Not intended for production deployment without additional hardening (auth, rate limiting, monitoring)
- Single-language interface (English only)

18. Future Recommendations

The following enhancements are recommended if the project advances beyond POC stage:

Area	POC Approach	Production Recommendation
Provider selection	Static ordered array	Dynamic selection based on real-time quota/latency metrics
Video processing	In-process FFmpeg	Background job queue (Bull, BullMQ)
File storage	Temporary local disk	Cloud storage (S3) with signed upload URLs
Rate limiting	None (provider limits)	Application-level rate limiting per user/IP
Caching	None	Cache by input hash to reduce redundant API calls
Monitoring	console.warn logging	Structured logging (Datadog, Sentry)
Multi-provider scoring	First success wins	Aggregate multiple providers for higher confidence
Format support	JPG/PNG/MP4 only	WebP, GIF, AVI, MOV, WEBM
Authentication	None	OAuth 2.0 or API key authentication
Internationalization	English only	i18n framework for multi-language support

— End of Document —